

Module 5: Problem Solving and Abstraction

CPE311 - Computational Thinking and Python

Prepared by Engr. Roman M. Richard

Introduction

- Which is easier to solve... big problems or small problems?
- If you want to solve small problems, what are the advantages?
- How do we get small problems?

Common Mistakes

- When problem solving, some common mistakes are:
 - Diving straight into the details;
 - Lack of consideration for problem decomposition.

Example: Tic-Tac-Toe

- If you would like to build a **Tic-Tac-Toe** game, what would be your considerations so that a human can play vs another human?
- Perform **problem decomposition**.

Example: Tic-Tac-Toe

- Problem decomposition can be performed by considering that we need to perform the following tasks:
 - Display the board
 - Make a move
 - Decide a winner
- Can these tasks/problems be divided further? Solved? Tested?

Deciding the Winner

- This task seems complicated, so let's decompose!
- Two methods for decomposition:
 - Top-Down Decomposition
 - Bottom-Up Decomposition

Top-Down Decomposition

Decompose	Make a Move	Display the Board
- Check 3 rows; - Check 3 columns; - Check 3 diagonals	- Ask for a move - Update the board	

- Can they be independently? Solved? Tested?
- Consideration: **When to stop decomposing a problem?**

When you are confident the small problems are solvable!

Tic-Tac-Toe: Make a move

- Recall there will be two human players, who makes a move?
- Do we want to:
 - Break it down into smaller problems: 1 for each player.
 - One problem: flexible for both players.
- **Prefer one problem that is flexible for 2 situations.** Always try to solve similar problems together rather than generating new similar problems.

Tic-Tac-Toe: Make a move

- How to solve with 1 problem? Create function **makeMove(x)** which can be used by both players.
- Problem abstraction? Performed through the parameters.

Summary

- Problem Decomposition:
 - Top-Down: decompose until you're confident you can solve the problems correctly.
- Problem Abstraction:
 - Different levels of abstraction in top-down decomposition from more general to specific.
 - Increase abstraction through the use of parameters. Avoid repeatedly writing and rewriting the same instructions for similar problems.

Seatwork: Top-Down vs Bottom-Up

- Today we discussed the Top-Down approach, but it is not the first time we have seen such algorithms.
- Discuss how the Tic-Tac-Toe game could be solved using a Bottom-Up approach.
- Post your answers under discussion 5.1.